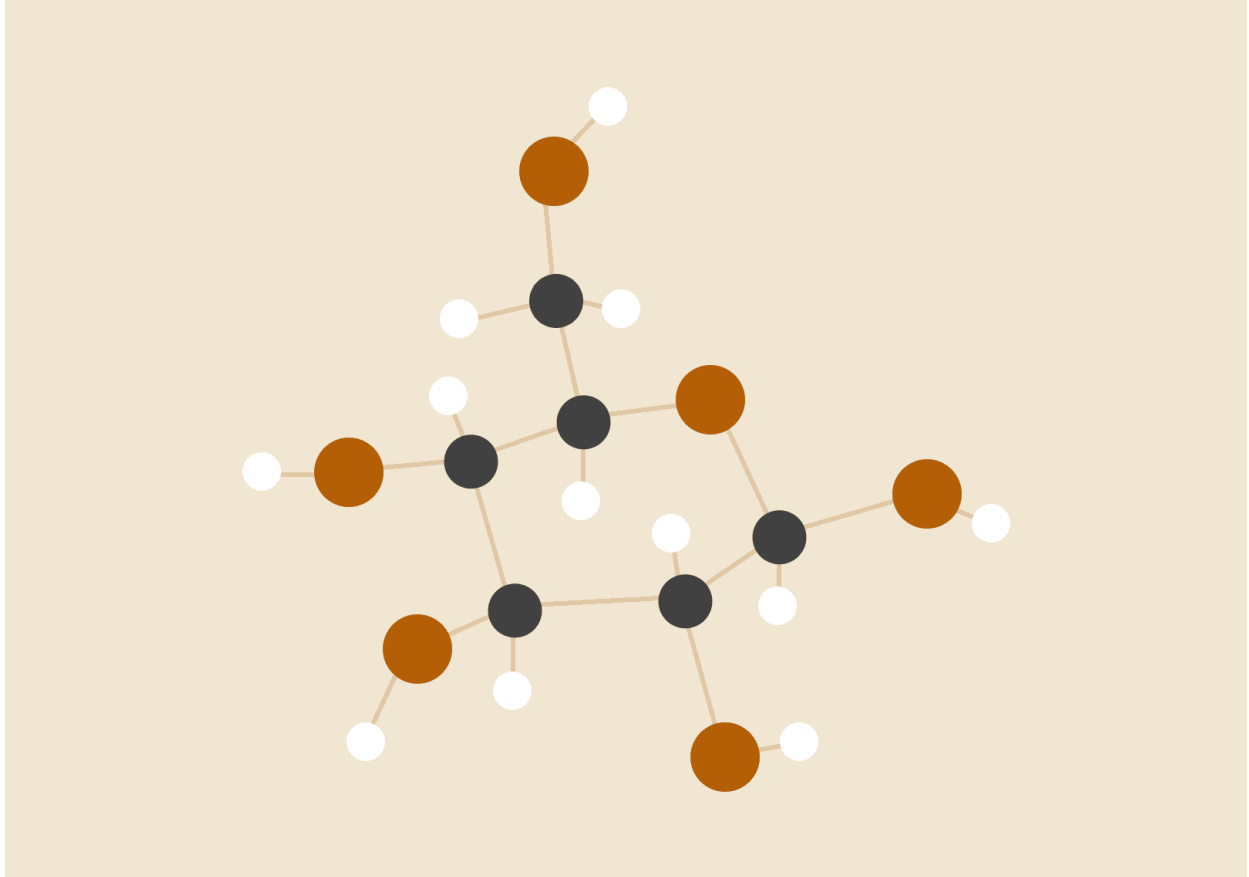


RAPPORT

Déploiement d'un LLM open-source avec Docker Compose et Kubernetes



Aissatou Sow

11/08/2026

Devops– DIC2 GIT

Introduction

Ce projet consiste à déployer une application composée de deux services — Ollama (moteur d'inférence LLM) et Open WebUI (interface web) — d'abord avec Docker Compose, puis à migrer ce déploiement vers Kubernetes en respectant les bonnes pratiques d'orchestration de conteneurs (gestion des ressources, health checks, stockage persistant, configuration externalisée).

Rôle de chaque ressource Kubernetes

- **Namespace (llm-devops)** : Isole toutes les ressources du projet dans un espace de noms dédié, évitant les collisions de noms avec d'autres applications du cluster et simplifiant la gestion/suppression groupée des ressources.
- **ConfigMaps (ollama-config, webui-config)** : Externalisent la configuration hors des images de conteneurs. ollama-config définit l'hôte d'écoute, le chemin des modèles, le délai de conservation en mémoire (OLLAMA_KEEP_ALIVE), le nombre maximum de modèles chargés simultanément (OLLAMA_MAX_LOADED_MODELS) et le délai maximal de chargement d'un modèle (OLLAMA_LOAD_TIMEOUT). webui-config fournit à Open WebUI l'URL du service Ollama à contacter. Séparer la config du code permet de modifier le comportement de l'application sans reconstruire les images.
- **PersistentVolumeClaims (ollama-pvc : 30Gi, webui-pvc : 2Gi)** : Réservent du stockage persistant indépendant du cycle de vie des pods (voir section 4).
- **Deployments (ollama, open-webui)** : Décrivent l'état désiré de chaque application (nombre de répliques, image, ressources, probes) et garantissent que Kubernetes recrée automatiquement un pod s'il tombe en panne. Le Deployment open-webui inclut un initContainer (wait-for-ollama) qui interroge `http://ollama:11434/api/version` en boucle avant de démarrer le conteneur principal — c'est l'équivalent Kubernetes du `depends_on`: condition: `service_healthy` de Docker Compose, qui n'existe pas nativement dans l'API Kubernetes.
- **Services (ollama, open-webui, type ClusterIP)** : Fournissent une adresse réseau stable et un nom DNS interne à chaque application, indépendamment des redémarrages ou changements d'adresse IP des pods (voir section 3).

- **Ingress (webui-ingress)** : Expose Open WebUI à l'extérieur du cluster via un nom d'hôte (llm.local), routé par le contrôleur ingress-nginx. C'est le point d'entrée HTTP unique du cluster, avec des annotations augmentant les délais de proxy (proxy-read-timeout) pour tolérer les réponses lentes des modèles.

Communication entre les services

Chaque pod reçoit une adresse IP interne au cluster, mais cette IP change à chaque création du pod (crash, mise à jour, redémarrage). Un Service Kubernetes résout ce problème en fournissant une IP virtuelle stable et un nom DNS interne (ollama.llm-devops.svc.cluster.local, raccourci en ollama au sein du même namespace) qui redirige automatiquement le trafic vers les pods correspondants via leurs labels (selector: app: ollama).

Dans notre déploiement : Open WebUI est configuré (via le ConfigMap webui-config) pour contacter Ollama à l'adresse `http://ollama:11434` — exactement la même logique que `OLLAMA_BASE_URL: http://ollama:11434` dans le `docker-compose.yaml` d'origine, où Docker Compose résout ollama via son propre DNS interne au réseau llm. Le trafic entrant, lui, passe par la chaîne : navigateur → Ingress Controller (nginx) → Service open-webui → pod Open WebUI.

Pourquoi les PersistentVolumeClaims sont nécessaires

Les conteneurs sont éphémères par nature : leur système de fichiers est perdu à chaque suppression ou création du pod (mise à jour d'image, crash, changement de configuration déclenchant un rollout). Sans stockage persistant, chaque redémarrage d'Ollama obligerait à retélécharger tous les modèles (plusieurs Go chacun), et chaque redémarrage d'Open WebUI effacerait les comptes utilisateurs et l'historique des conversations.

Un PVC découple le stockage du cycle de vie du pod : il réserve un volume qui survit à la suppression et à la création du pod. `ollama-pvc` (30Gi) conserve les poids des modèles téléchargés sous `/root/.ollama` ; `webui-pvc` (2Gi) conserve la base de données SQLite d'Open WebUI (comptes, conversations, configuration) sous `/app/backend/data`. Nous avons d'ailleurs vérifié concrètement ce comportement pendant le projet : après une

suppression complète et une recreation du cluster minikube, les modèles ont pu être restaurés depuis une sauvegarde du volume sans avoir à les retélécharger, illustrant que la persistance est bien découplée du pod et du nœud applicatif (bien que liée, dans notre cas, au nœud minikube lui-même).

Rôle des probes (Startup, Readiness, Liveness)

Les trois probes servent des objectifs différents et se complètent :

- **Startup Probe** : Protège les applications à démarrage lent (chargement de modèles, migrations de base de données) contre une terminaison prématurée par la liveness probe. Tant qu'elle n'a pas réussi, les autres probes sont suspendues. Configurée ici avec une tolérance large (jusqu'à 300s) car Ollama et Open WebUI peuvent mettre du temps à devenir disponibles au premier démarrage.
- **Readiness Probe** : Détermine si un pod est prêt à recevoir du trafic. Un pod qui échoue à sa readiness probe est retiré des endpoints du Service (mais n'est pas redémarré) — utile par exemple si Ollama est temporairement surchargé pendant le chargement d'un modèle volumineux et ne doit pas recevoir de nouvelles requêtes.
- **Liveness Probe** : Détermine si un pod est bloqué ou dans un état irrécupérable. Un échec répété entraîne le redémarrage automatique du conteneur par Kubernetes (auto-guérison).

(Lors du déploiement, nous avons observé un cas où la liveness probe d'Ollama échouait car le pod était surchargé en mémoire (plusieurs modèles chargés simultanément faute d'un OLLAMA_KEEP_ALIVE trop long), provoquant un redémarrage en boucle. Ce n'est pas la probe qui était en cause, mais elle a joué son rôle correctement en détectant un pod réellement non-fonctionnel — la vraie correction a été de limiter la consommation mémoire côté application)

Docker Compose vs Kubernetes

ASPECT	DOCKER COMPOSE	KUBERNETES
Portée	Un seul hôte Docker	Cluster de plusieurs noeuds

Auto-guérison	Redémarre un conteneur planté sur un même hôte (restart : unless-stopped)	Reprogramme un pod sur n'importe quel noeud sain, avec réconciliation
Mise à l'échelle	Manuelle(docker compose up -- scale), limitée aux ressources de l'hôte	Horizontale et automatisable (HPA), répartie sur plusieurs nœuds
Dépendance entre services	depends_on avec condition : service_healthy, natif et simple	Aucun mécanisme natif équivalent — nécessite un initContainer ou une logique de retry côté application
Découverte de services	DNS interne au réseau bridge défini dans le compose file	DNS interne au cluster via les Services, indépendant du noeud physique
Configuration	Variables d'environnement et fichier .env	ConfigMaps et Secrets, montables en variables d'env ou en fichiers
Cas d'usage type	Développement local, démonstration, petits environnements mono-machine	Production, haute-disponibilité, montée en charge, environnements multi-machines

En résumé, Docker Compose reste plus simple et suffisant pour du développement local mono-machine, tandis que Kubernetes apporte l'orchestration, la résilience et la scalabilité nécessaires à un déploiement de production distribué — au prix d'une complexité de configuration nettement supérieure.